

Algoritmer & Søgning — Eksamens Snydeark

Datamatiker EAAA · Java · Programmering

1. Insertion Sort

Idé: Tag ét element ad gangen og skub det ind på den rigtige plads blandt de allerede sorterede elementer — ligesom at sortere spillekort i hånden.

Visuelt eksempel:

[299, 149, 449, 99]

Tag 149 → 299 > 149 → skub 299 → [149, 299, 449, 99]

Tag 449 → 299 < 449 → bliv → [149, 299, 449, 99]

Tag 99 → skub 449, 299, 149 → [99, 149, 299, 449]

Kode:

```
public static void sort(int[] arr) { for (int i = 1; i < arr.length; i++) { int key = arr[i]; // tag elementet int j = i - 1; // start sammenligning bagud while (j >= 0 && arr[j] > key) { arr[j + 1] = arr[j]; // skub frem j--; // gå bagud } arr[j + 1] = key; // placer på rette plads } }
```

■ **HUSKEREGEL:** $j = i - 1$ og $j--$ — du går altid BAGUD. Hvis du skriver $j++$ er det forkert!

Big O: $O(n^2)$ i værste tilfælde · $O(n)$ hvis næsten sorteret

2. Merge Sort (Del og Kombiner)

Idé: Del arrayet i to halvdele, sorter hver halvdel rekursivt, flet dem sammen. Basecase: array med 1 element er allerede sorteret.

Visuelt eksempel:

[64, 34, 25, 12]

Del → [64, 34] og [25, 12]

Del → [64] [34] og [25] [12]

Flet → [34, 64] og [12, 25]

Flet → [12, 25, 34, 64]

Kode:

```
import java.util.Arrays; public static int[] sort(int[] arr) { if (arr.length <= 1) return arr; // BASECASE int mid = arr.length / 2; int[] left = Arrays.copyOfRange(arr, 0, mid); int[] right = Arrays.copyOfRange(arr, mid, arr.length); left = sort(left); // rekursivt right = sort(right); // rekursivt return merge(left, right); } private static int[] merge(int[] left, int[] right) { int[] result = new int[left.length + right.length]; int i = 0, j = 0, k = 0; while (i < left.length && j < right.length) { if (left[i] <= right[j]) result[k++] = left[i++]; else result[k++] = right[j++]; } while (i < left.length) result[k++] = left[i++]; while (j < right.length) result[k++] = right[j++]; // NB: right ikke left! return result; }
```

■ **HUSKEREGEL:** Sidste while-løkke bruger $right[j++]$ — IKKE $left[j++]$!

Big O: $O(n \log n)$ — deler $\log n$ gange, hvert niveau koster n sammenligninger

3. Lineær Søgning

Idé: Gennemgå hvert element ét ad gangen indtil target er fundet. Virker på usorterede arrays.

Kode:

```
public static int linear(int[] arr, int target) { for (int i = 0; i < arr.length; i++) { if (arr[i] == target) return i; // fundet } return -1; // ikke fundet }
```

Big O: $O(n)$ — i værste tilfælde gennemgås alle elementer

4. Binær Søgning

Idé: Halver søgeområdet ved hvert trin. Kræver et SORTERET array!

Visuelt eksempel:

Array: [99, 149, 199, 249, 299, 349, 449] · Søger: 199

mid=3 → arr[3]=249 → 199 < 249 → søg venstre (high=2)

mid=1 → arr[1]=149 → 199 > 149 → søg højre (low=2)

mid=2 → arr[2]=199 → FUNDET på index 2 ✓

Kode:

```
public static int binary(int[] arr, int target) { int low = 0; int high = arr.length - 1; while (low <= high) { int mid = (low + high) / 2; if (arr[mid] == target) return mid; // fundet! else if (arr[mid] < target) low = mid + 1; // søg højre else high = mid - 1; // søg venstre } return -1; // ikke fundet }
```

■ **HUSKEREGEL:** low, high, mid — halvér hver gang. Kræver sorteret array!

Big O: $O(\log n)$ — meget effektiv på store datasæt

5. Big O Oversigt

Algoritme	Big O	Kræver sorteret?	Bemærkning
Insertion Sort	$O(n^2)$	Nej	God til næsten-sorterede lister
Merge Sort	$O(n \log n)$	Nej	Effektiv, del og kombiner
Lineær søgning	$O(n)$	Nej	Simpel, gennemgår alt
Binær søgning	$O(\log n)$	JAJ!	Meget hurtig, men kræver sortering